



哈爾濱工業大學(深圳)
HARBIN INSTITUTE OF TECHNOLOGY, SHENZHEN

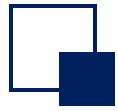
規格嚴格 功夫到家
1920 — 2017

Chapter 4 Interfaces and Inheritance



Outline

- **Inheritance**
- **Interfaces and abstract classes**
- **Polymorphism and rewriting**
- **Java multi-inheritance issues**
- **Superclass vs. super keyword**
- **Inheritance framework for exceptions**



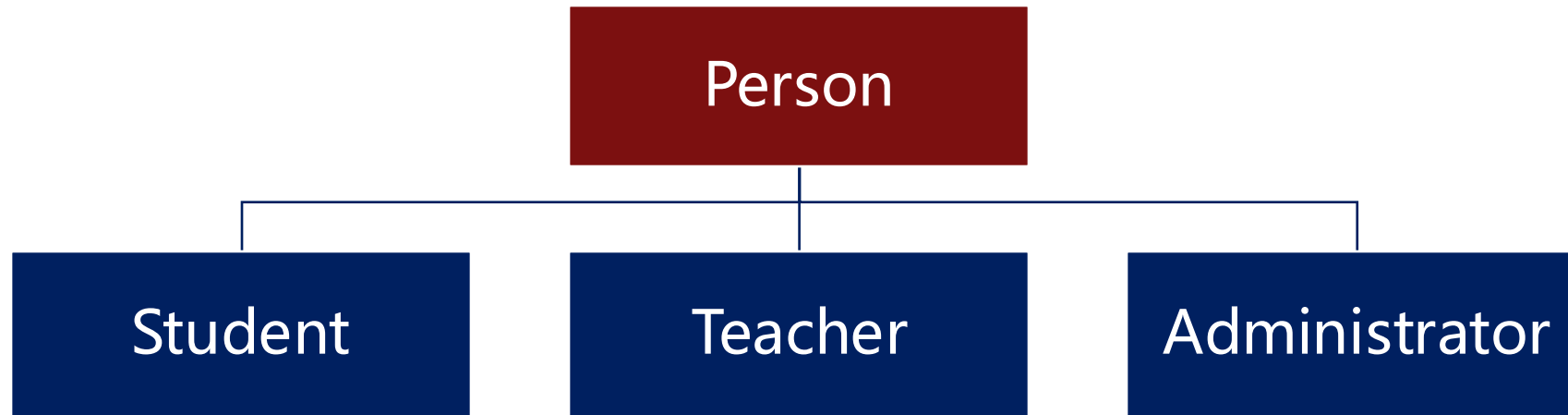
Why Inheritance?

```
class Person {  
    String name;  
    int age;  
    public String getName() {...}  
    public void setName(String name) {...}  
}
```

```
class Student {  
    String name;  
    int age;  
    String school;  
    public String getName() {...}  
    public void setName(String name) {...}  
    public String getSchool() {...}  
}
```

Lots of duplicate code!

Inheritance - Superclass and subclasses

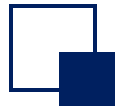


- ❑ The Person class is related to other classes, such as the Student class
- ❑ There is an obvious is-a relationship between Student and Person, which is a clear feature of inheritance



Inheritance - classes, superclasses, and subclasses

- ❑ The keyword “extends” indicates that a new class being constructed derives from an existing class.
 - Existing classes: superclass, base class, parent class
 - New classes: subclasses, derivatives, children
- ❑ Inheritance reuses the methods of existing classes and adds new methods and fields to adapt the new classes to new situations.
- ❑ Parent/superclasses describe the properties and methods that are shared by the subclasses.
- ❑ Different subclasses have their own different properties and methods



Inheritance - Defines superclass, subclass

```
1. public class Person{
2.     private String name;
3.     private int age;
4.     public Person(String name, int age){
5.         this.name = name;
6.         this.age = age;
7.     }
8.     public String getName(){
9.         return name;
10.    }
11.    private String setName (String name){
12.        this.name = name;
13.    }
14. }
```

```
1. public class Student extends Person{
2.     String school;
3.     public Student(String name, int age, String school) {
4.         super(name, age); // Call the constructor of the
                           superclass
5.         this.school= school;
6.     }
7.
8.
9.     public string getSchool(){
10.        return this.school;
11.    }
12. }
```

- Make use of the extends keyword
- The child class cannot directly inherit the constructor of the parent class, and needs to use the super keyword
- Subclasses can reuse the properties and methods of the parent class, and subclasses can add some new properties and methods



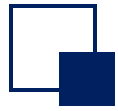
Inheritance

- ❑ Inheritance is one of the three important features of object-oriented programming
- ❑ Advantages
 - Improve code reusability
 - Improve the scalability of the program
 - This creates a relationship between classes and forms the basis of polymorphism
- ❑ Disadvantages
 - Increase class coupling (changes in one class affect other related classes)

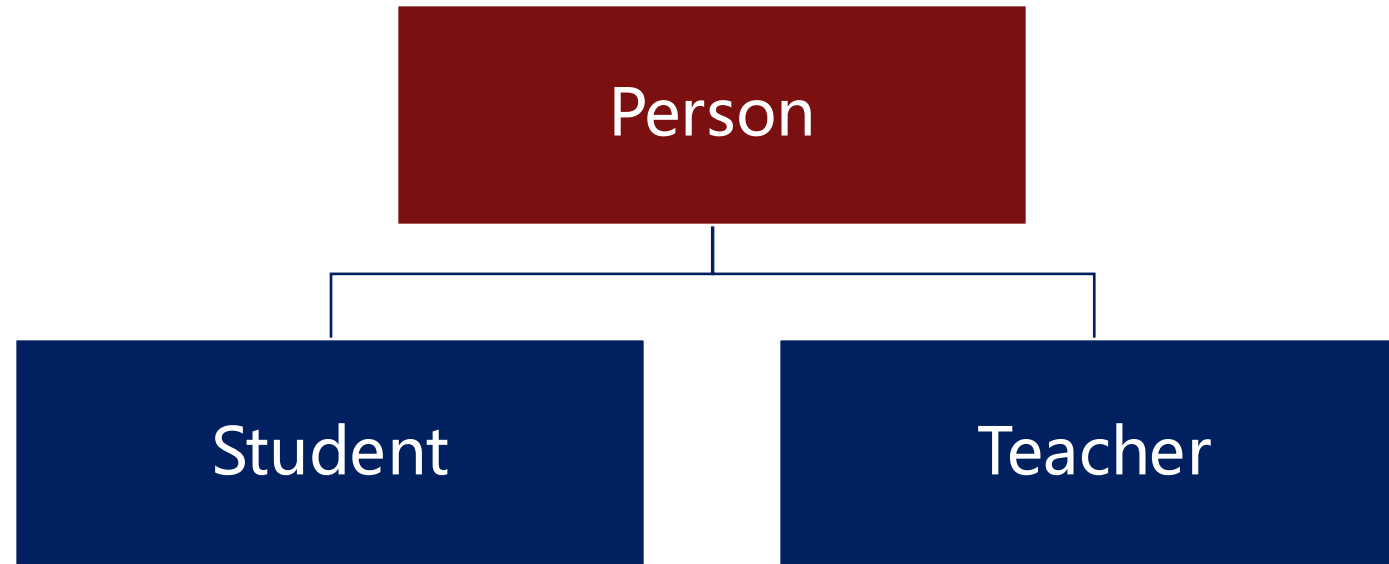


Outline

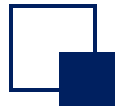
- Inheritance
- Interfaces and abstract classes
- Polymorphism and rewriting
- Java multi-inheritance issues
- Superclass vs. super keyword
- Inheritance framework for exceptions



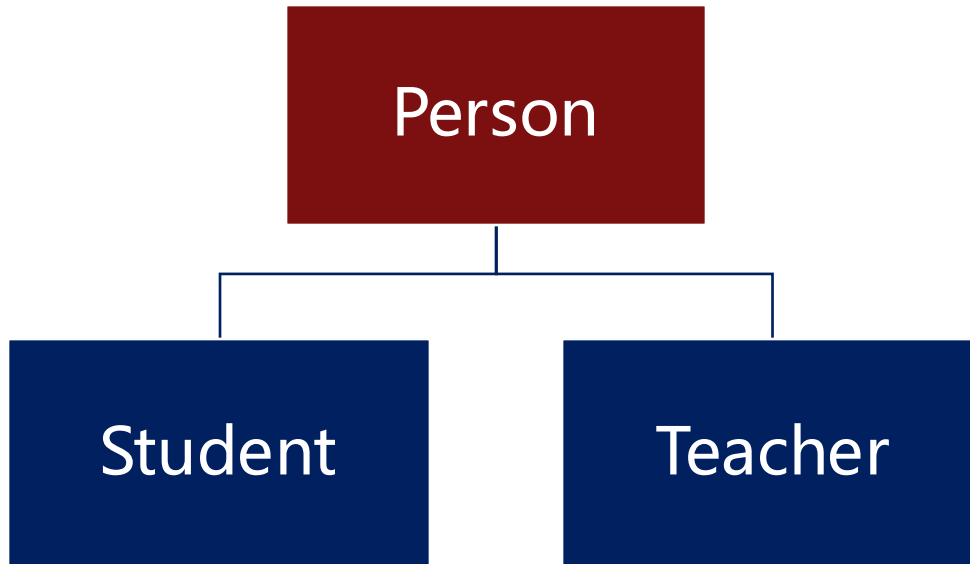
Abstract Class



- The parent class extracts the properties and methods that are common to the child classes.
- If any of these properties and methods are undecided, then we can define them as abstract methods and implement them in the later subclasses.
- To put it simply, a class with an abstract method is an abstract class.



Abstract Class - Define an abstract class



```
1. //Define an abstract class
2. public abstract class Person{
3.     //Ordinary method
4.     public String getName()
5.     {
6.         return name;
7.     }
8.     //Abstract approach
9.     //There is no method body, and it
    is decorated with abstract
10.    public abstract void getMission();
11.}
12.
13. Person P = new Person();
14. // Compilation error, abstract class
    cannot be instantiated
```



Abstract Class - A subclass of an abstract class

- Abstract methods act as placeholder methods and are implemented concretely in subclasses

```
public class Student extends Person{  
    public void getMission(){  
        System.out.println("Studying");  
    }  
}
```

```
public class Teacher extends Person{  
    public void getMission(){  
        System.out.println("Teaching");  
    }  
}
```

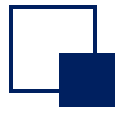


Abstract Classes - The role of abstract classes

❑ Abstract classes cannot be instantiated

❑ function:

- An abstract approach is actually equivalent to defining a "specification"
- It can only be inherited, and the subclass implements its defined abstract methods
- Can be used to implement polymorphism



Interface

- ❑ In abstract classes, the abstract method essentially defines the interface specification to ensure the subclasses all have the same interface implementation
- ❑ Interface:
 - More abstract than an abstract class: without attributes, all methods are abstract

```
1. //Define an abstract class
2. public abstract class Person{
3.     //Abstract method
4.     public abstract void getDuty();
5.     public abstract void getMission();
6. }
```

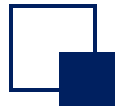
```
1. //Define an interface
2. interface Person{
3.     //Abstract method
4.     void getDuty();
5.     void getMission();
6. }
```



Interface

- ❑ When a specific class implements an interface, the implements keyword is used

```
public class Student implements Person{
    private String name;
    private int age;
    @Override
    public void getDuty(){
        System.out.println("Study");
    }
    @Override
    public void getMission(){
        System.out.println("Study hard");
    }
}
```



Interface VS. Abstract classes

□ Interface vs. abstract class

	Abstract class	Interface
Abstract method	Abstract methods can define	Abstract methods can define
Field	Abstract methods can define	No fields
Inheritance	Can only extends one class	Can implements many interfaces

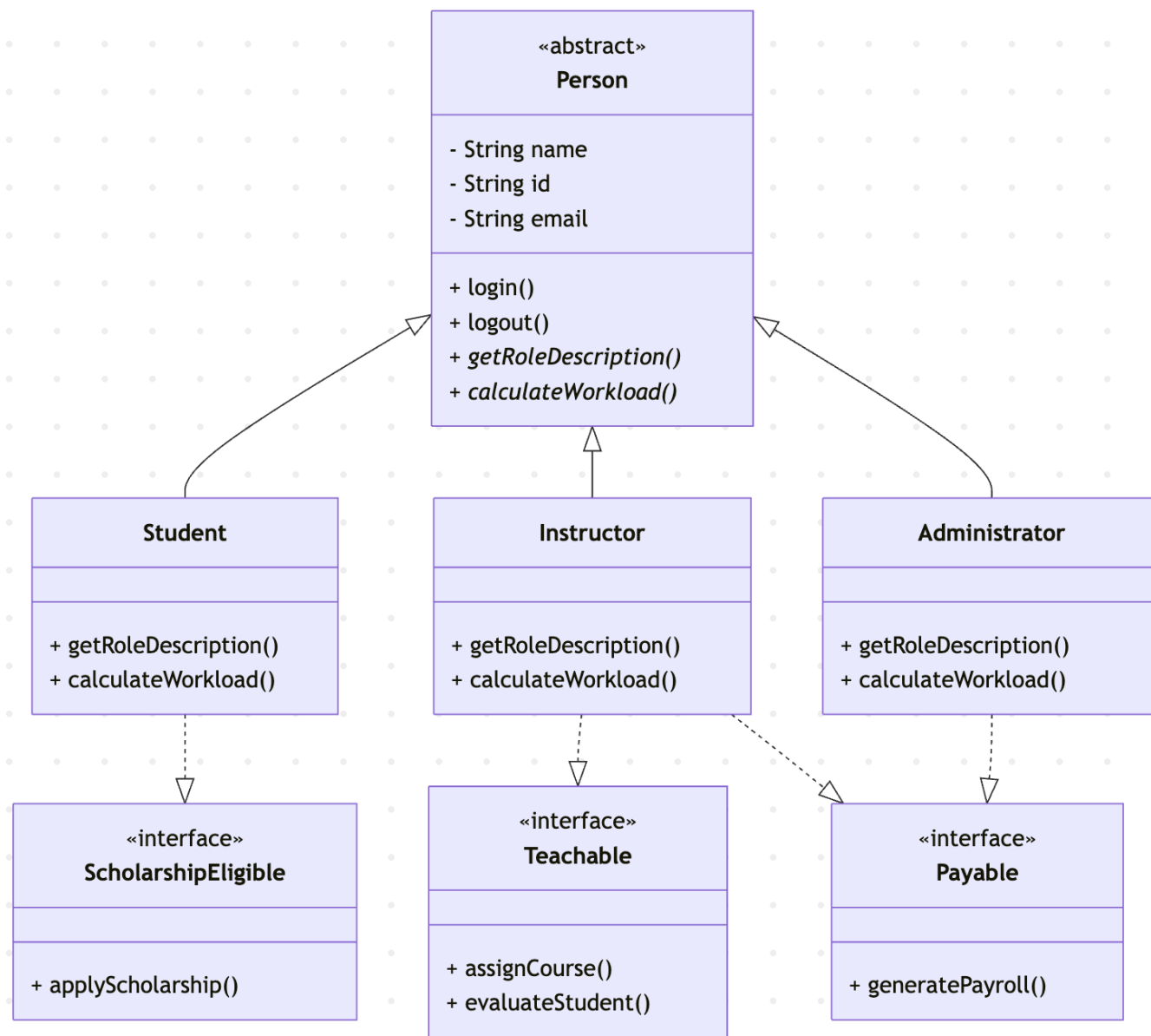
□ Why do you need interfaces when you have abstract classes?

- Abstract classes don't solve the problem of multiple inheritance



Question

- You are designing a Student Management System with three types of people: Student, Instructor, and Administrator. Each has basic information (name, id, email) and actions like login() and logout(). Some people also have extra roles — a student can apply for scholarships, an instructor can teach and get paid, and an administrator gets paid.
- How would you design this system to reuse common data and behaviors while keeping flexibility for special roles using abstract class, interfaces, or both?





Outline

- Inheritance
- Interfaces and abstract classes
- **Polymorphism and Override**
- Java multi-inheritance issues
- Superclass vs. super keyword
- Inheritance framework for exceptions



Polymorphism

- ❑ Polymorphism is one of the three important features of object-oriented programming
- ❑ Polymorphism is the ability of the same behavior to have multiple manifestations
 - Different ways to deal with it in different situations
 - The variables and methods defined in a program are not determined at the time of programming, but only during the program's run.



Polymorphism

❑ Benefits

- Reduce coupling
- Enhance replaceability
- Enhance scalability
- Increase flexibility

❑ Three necessary conditions:

- inherit
- override
- The parent class reference
points to the child class

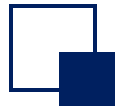
❑ Three ways to implement polymorphism:

- **Override**
- **Abstract Classes and Abstract Methods**
- **Interface**



Implementation 1: Override

- ❑ Some methods in the parent class don't necessarily work for the child class. This method needs to be overridden in the parent class.
- ❑ Override: If you define a method in a child class with the same name, parameters, and return type as a method in the parent class, then you can say that the method of the child class overrides the method of the parent class.



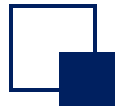
Implementation 1: Override

```
1. public class Person{
2.     private String name;
3.     private int age;
4.     public void getTarget(){
5.         System.out.println( "The good
   life" );
6.     }
7. }
```

```
1. public class Student extends Person{
2.     @Override
   public void getTarget(){
3.         System.out.println("Learn");
4.     }
5. }
```

```
1. public class Teacher extends Person{
2.     @Override
   public void getTarget(){
3.         System.out.println("Teach");
4.     }
5. }
```

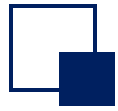
- In the Student and Teacher classes, we've overridden the getTarget method to override the previous getTarget.
- These methods have the same name, return type, and parameters.
- Programming skills: add @Override allows the compiler to help check if the correct override has been made.



Implementation 1: Override

```
public static void show_target(Person P_x) {  
    System.out.print(" target: ");  
    P_x.getTarget();  
}  
Person P1= new Student();  
Person P2= new Teacher();  
show_target(P1);  
show_target(P2);
```

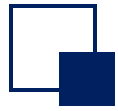
- The method of the child class override is invoked by reference to the parent class
- In the show_target() function, we only need to pass in a reference to the parent class
 - **Super_class_name reference_name = new subclass_name();**



Implementation 2: abstract classes



```
1. //Define an abstract class
2. public abstract class Person{
3.     .....
4.     //Abstract approach
5.     //There is no method body, and it
   is decorated with abstract
6.     public abstract void getTarget();
7.     .....
8. }
```

Implementation 2: abstract classes

```
public class Engineer extends Person{
    @Override
    public void getTarget(){
        System.out.println("Technology
changes the world");
    }
}
```

```
public class Scientist extends Person{
    @Override
    public void getTarget(){
        System.out.println("Climb the
heights of science");
    }
}
```

```
public static void main(String[] args) {
    Person Wang = new Engineer();
    Wang.getTarget();
    Person Li = new Scientist();
    Li.getTarget();
}
```

- Definition format for polymorphism:
- Superclass_name reference_name = new subclass_name ();

`Person Wang = new Engineer();`

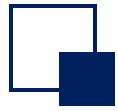


Implementation 3: interfaces



```
//Interface: Target  
public interface GetTarget(){  
    void getTarget();  
}
```

An interface is a pure abstract class that is more abstract than a normal abstract class, with no attributes, only methods.



Implementation 3: interfaces

```
//Talent class implements the target interface
```

```
public class Talent implements GetTarget(){
```

```
    @Override
```

```
    public void getTarget(){
```

```
        System.out.println("Build the most important facilities for  
the country");
```

```
    }
```

```
}
```

```
// Entrepreneur class implements the target interface
```

```
public class Entrepreneur implements GetTarget(){
```

```
    @Override
```

```
    public void getTarget(){
```

```
        System.out.println("Promote common prosperity");
```

```
    }
```

```
}
```



Implementation 3: interfaces

```
public static void main(String[] args) {  
    GetTarget Liu = new Talent();  
    Liu.getTarget();  
    GetTarget Ma = new Entrepreneur();  
    Ma.getTarget();  
}
```

The objects Liu and Ma here can be considered as subclasses of the interface GetTarget

Which of the following descriptions is **wrong**?

- ☐ A Inheritance is the basis of polymorphism
- ☐ B Inheritance is good for reducing the coupling of classes, and changes in one class do not affect other classes
- ☐ C Polymorphism can increase the flexibility of the program
- ☐ D There are no attribute fields in the interface, and abstract methods can be defined to support multiple inheritance

Which of the following descriptions is wrong?

- ☐ A Inheritance is the basis of polymorphism
- ☒ B Inheritance is good for reducing the coupling of classes, and changes in one class do not affect other classes
- ☐ C Polymorphism can increase the flexibility of the program
- ☐ D There are no attribute fields in the interface, and abstract methods can be defined to support multiple inheritance



```
abstract class Person {
    String name;
    Person(String name) { this.name = name; }

    public String describe() {
        return "Person: " + name;
    }

    public abstract int weeklyWorkload();
}

class Student extends Person {
    int credits;
    Student(String name, int credits) { super(name); this.credits = credits; }

    @Override
    public String describe() { return "Student: " + name; }

    @Override
    public int weeklyWorkload() { return credits * 3; }

    public String describe(String prefix) { return prefix + " Student: " + name; }
}

class Instructor extends Person {
    int courses;
    Instructor(String name, int courses) { super(name); this.courses = courses; }

    @Override
    public String describe() { return "Instructor: " + name; }

    @Override
    public int weeklyWorkload() { return 10 + courses * 5; }
}
```

```
class Administrator extends Person {
    String department;
    Administrator(String name, String department) { super(name); this.department = department; }

    @Override
    public String describe() { return "Administrator: " + name + ", dept: " + department; }

    @Override
    public int weeklyWorkload() { return 40; }
}

public class Demo {
    public static void main(String[] args) {
        Person[] people = {
            new Student("Alice", 4),
            new Instructor("Bob", 2),
            new Administrator("Carol", "Finance")
        };

        for (Person p : people)
            System.out.println(p.describe() + " | workload = " + p.weeklyWorkload());

        System.out.println("\nOverloading Example:");
        Student s = new Student("Dave", 5);
        System.out.println(s.describe());
        System.out.println(s.describe("Honor"));
    }
}
```



```
public class Demo {  
    public static void main(String[] args) {  
        Person[] people = {  
            new Student("Alice", 4),  
            new Instructor("Bob", 2),  
            new Administrator("Carol", "Finance")  
        };  
  
        for (Person p : people)  
            System.out.println(p.describe() + " | workload = " + p.weeklyWorkload());  
  
        System.out.println("\nOverloading Example:");  
        Student s = new Student("Dave", 5);  
        System.out.println(s.describe());  
        System.out.println(s.describe("Honor"));  
    }  
}
```

```
Student: Alice | workload = 12  
Instructor: Bob | workload = 20  
Administrator: Carol, dept: Finance | workload = 40
```

```
Overloading Example:  
Student: Dave  
Honor Student: Dave
```




Outline

- Inheritance
- Interfaces and abstract classes
- Polymorphism and Override
- **Java multi-inheritance issues**
- Superclass vs. super keyword
- Inheritance framework for exceptions

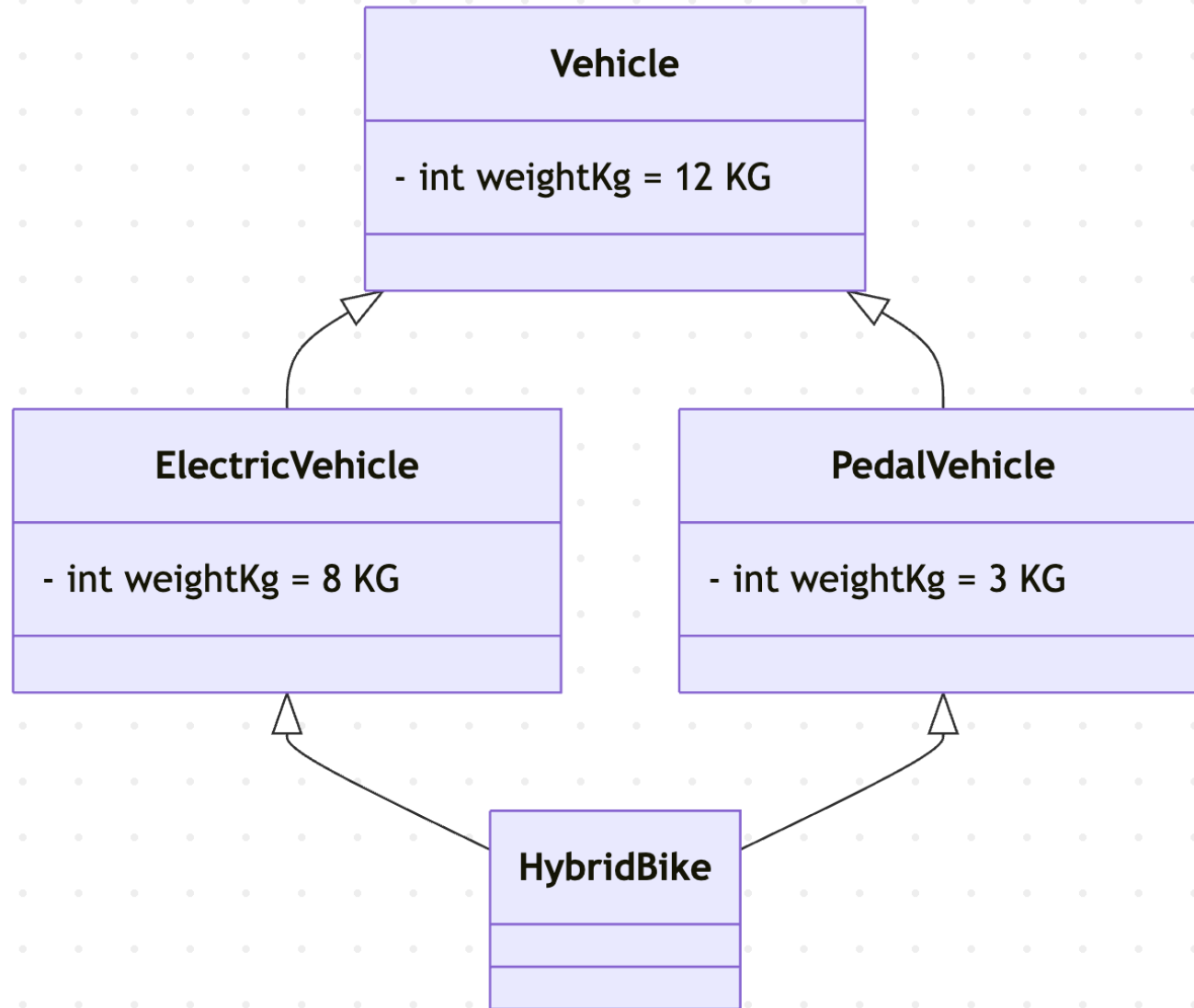


Does Java allow multiple inheritances

- ❑ Multiple inheritance means that a class can inherit behaviors and characteristics from multiple parent classes at the same time, whereas Java only allows single inheritance to ensure data security.
- ❑ Problems with multiple inheritance:
 - If a child class inherits from multiple parent classes that has the same member variable, the child class will not be able to tell which parent class's member variable to use when referencing the variable.
 - If a child class inherits from multiple parent classes that have the same method, and the child class does not override the method (if it does, it uses the method in the child class), then it will not be possible to determine which parent class's method is called when the method is called.

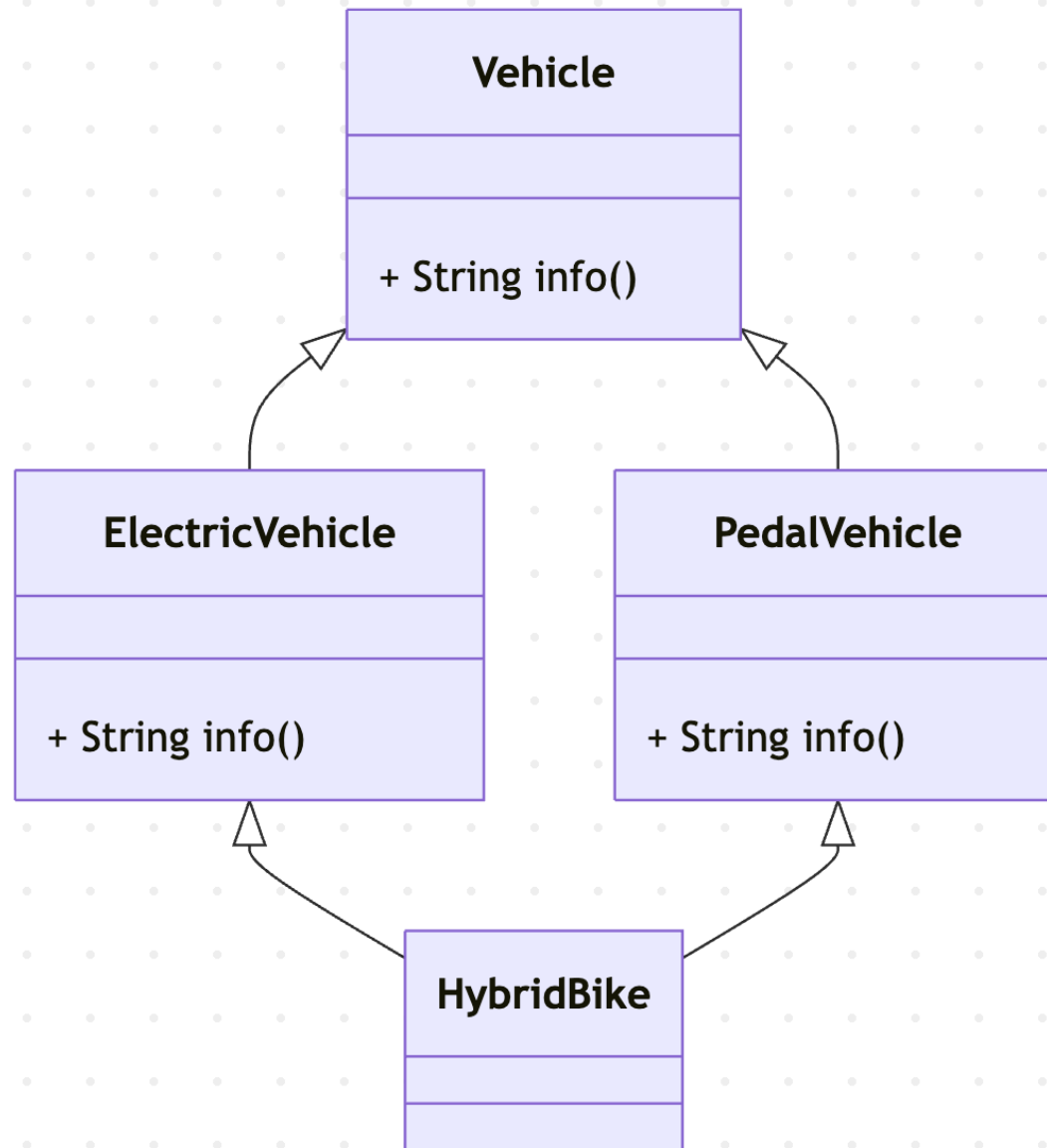


Diamond Problem





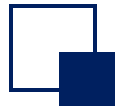
Diamond Problem





How Java implements multi-inheritance effects

- ❑ Sometimes developers do need to implement multiple inheritance, and this is the case in real life. Heredity, for example, we can inherit both the father's and the mother's behaviors and traits.
- ❑ Java provides two ways to implement multiple inheritance:
 - Inner classes
 - Interface



Multi-inheritance 1: Inner class

- ❑ Inner Classes: Inside a class, you can define not only member variables and methods, but also another class. If you define another class B inside class A, class B is called an inner class, and class A is called an outer class.
 - The inner classes are hidden from the outside world , and you have access to all elements in the outer class.



Multi-inheritance 1: Inner class implementation

- ❑ We will introduce how children can use internal classes to inherit the good genes of both their father and mother using common examples of heredity in their lives.
- ❑ We create the Father class, add the strong() method to the class; Again, create the Mother class and add the smart() method to the class, as follows:

```
public class Father{  
    public int strong(){  
        // Strength Index  
        return 9;  
    }  
}
```

```
public class Mother{  
    public int smart(){  
        // Intelligence Index  
        return 8;  
    }  
}
```



Multi-inheritance 1: Inner class implementation

Then, create a Son class in which you can implement multiple inheritance effects with an inner class. The code looks like this on the right:

➤ The Son class defines two inner classes that inherit from the Father class and the Mother class, respectively, and both get the behavior of their respective parent classes.

```
1. public class Son {
2.     // Inner class inherits from the Father class
3.     class Father_Inner extends Father {
4.         public int strong() {
5.             return super.strong() + 1;
6.         }
7.     }
8.     // Inner class inherits the Mother class
9.     class Mother_Inner extends Mother {
10.        public int smart() {
11.            return super.smart() + 2;
12.        }
13.    }
14.    public int getStrong() {
15.        return new Father_Inner().strong();
16.    }
17.    public int getSmart() {
18.        return new Mother_Inner().smart();
19.    }
20. }
```




Multi-inheritance 2: Implemented with interfaces

❑ First of all, define interfaces Father and Mother. And then, we can use Daughter to inherit from Father and Mother.

➤ In Java, a class can inherit only one class, but an interface can inherit multiple interfaces.

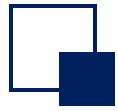
```
1. public interface Father{  
2.     public void strong();  
3. }  
4.  
5. public interface Mother{  
6.     public void smart();  
7. }  
8.  
9. public interface Daughter extends Father, Mother{  
10.     public void kind();  
11. }
```



Multi-inheritance 2: Implemented with interfaces

Then, the Girl class implements the interface Daughter, and rewrites the strong method of Father, the smart method of Mother, and the kind method of Daughter in the class.

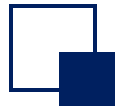
```
1. public class Girl implements Daughter{
2.     public static void main(String[] args){
3.     }
4.     @Override
5.     public void strong(){
6.         System.out.println("She's not strong.");
7.     }
8.     @Override
9.     public void smart(){
10.        System.out.println("She's very smart.");
11.    }
12.    @Override
13.    public void kind(){
14.        System.out.println("She's very kind.")
15.    }
16. }
```



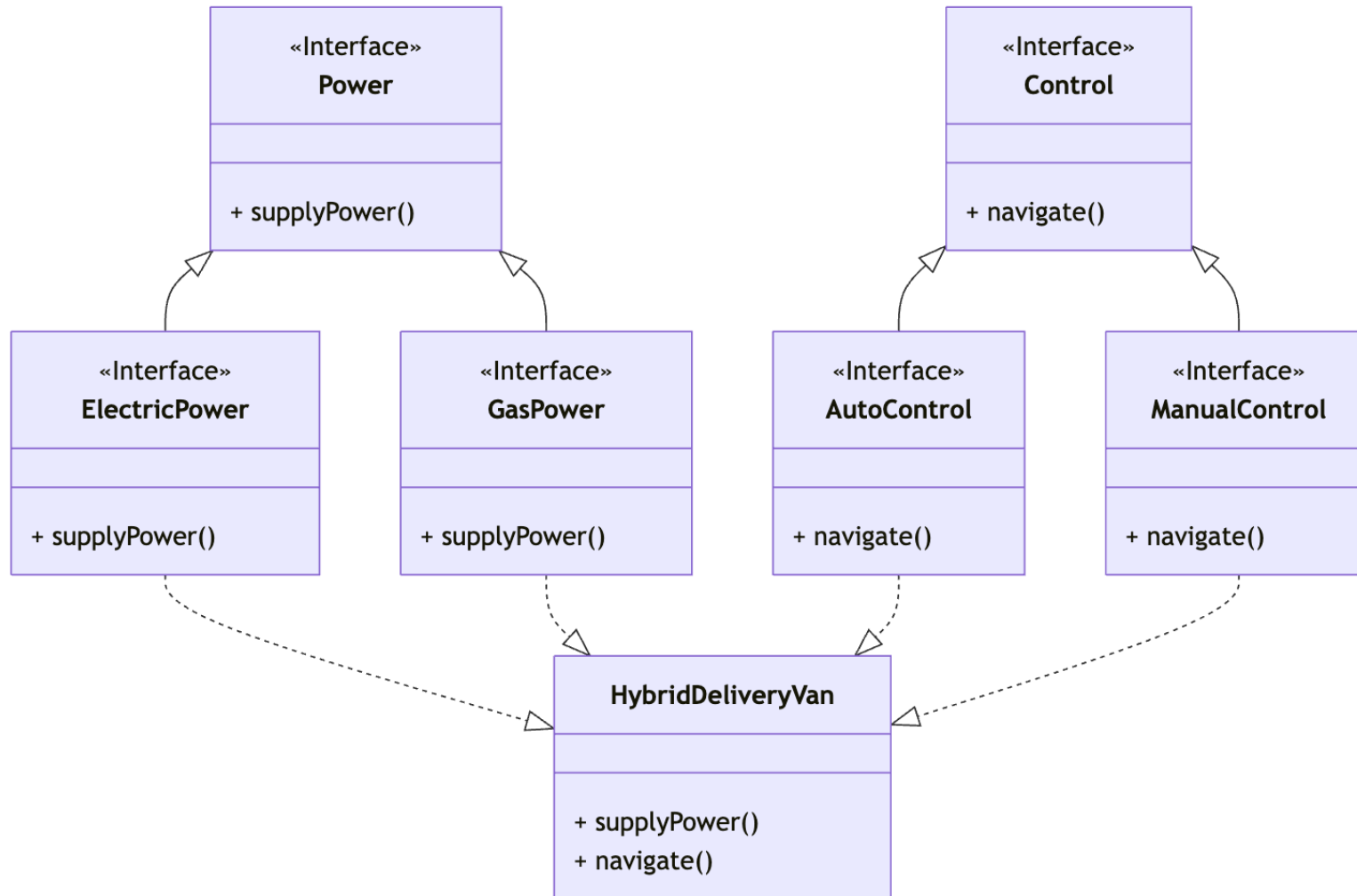
Quiz

Design a HybridDeliveryVan that uses both ElectricPower and GasPower, which are two kinds of Power, and supports both AutoControl and ManualControl, which are two kinds of Control.

Draw the class diagram.



Quiz





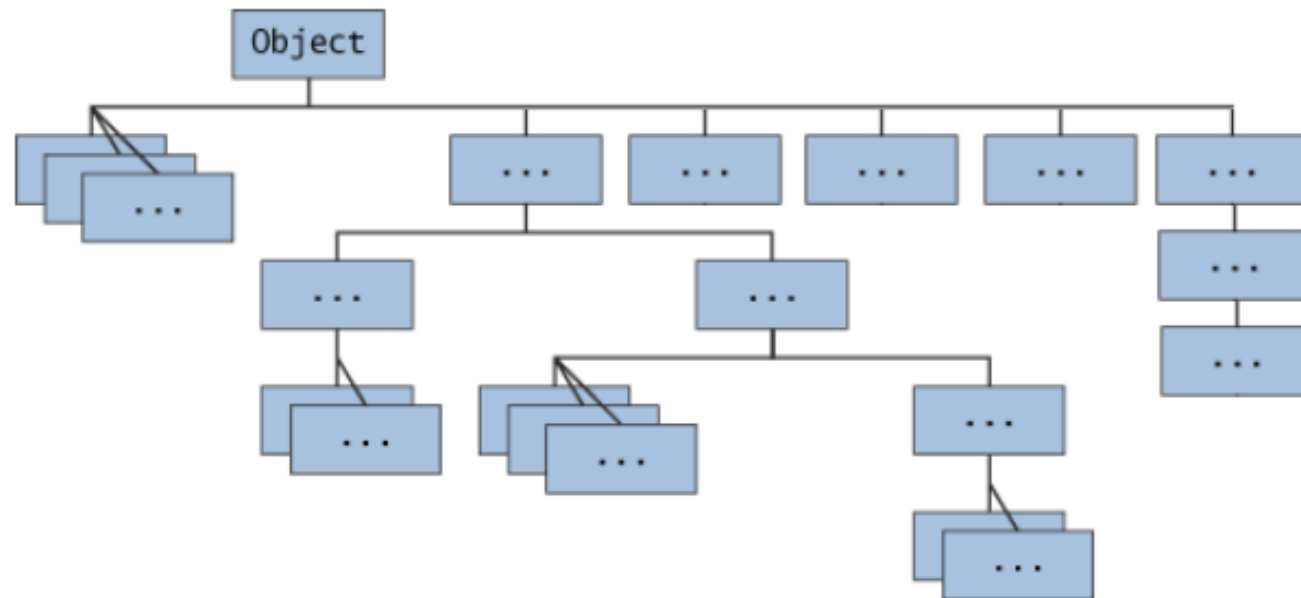
Outline

- Inheritance
- Interfaces and abstract classes
- Polymorphism and Override
- Java multi-inheritance issues
- **Superclass vs. super keyword**
- Inheritance framework for exceptions



Object superclass

- ❑ The Object class is the ancestor of all classes in Java, and in Java each class extends the Object.
- ❑ The Object class is in the java.lang package and is automatically imported at compile time.





Object superclass

■ The Object class can inherit explicitly or implicitly, either way :

➤ Explicitly inherit

```
1. public class Student extends Object{  
2.  
3. }
```

➤ Implicitly inherit

```
1. public class Student{  
2.  
3. }
```



Object superclass question?

- ❑ Can a class explicitly extend another class and also implicitly extend Object in Java?

```
1. public class Student extends Person {  
2.  
3. }
```

```
1. public class Person extends Object {  
2.  
3. }
```




Variables of the Object type

- ❑ Any object can be referenced with a variable of type Object:

```
Object obj = new Student("Zhangsan", 19);
```

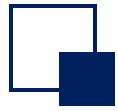
- ❑ Of course, a variable of type Object can only be used as a generic container for various values. In order to perform specific operations on the contents, you also need to know the original type of the object and perform the corresponding casting:

```
Student Zhangsan = (Student) obj;
```



Example

```
public class Demo1 {  
    public static void main(String[] args) {  
        List<Object> bag = new ArrayList<>();  
        bag.add("42");           // String  
        bag.add(42);             // Integer  
        bag.add(3.14);           // Double  
  
        // Somewhere else, a teammate assumes "it's a list of integers"  
        try {  
            int sum = 0;  
            for (Object o : bag) {  
                sum += (Integer) o; // ✨ ClassCastException on the first "42"  
            }  
            System.out.println(sum);  
        } catch (ClassCastException e) {  
            System.out.println("Boom: " + e);  
        }  
    }  
}
```



equals method

- ❑ The equals() method in the Object class is used to compare whether two objects are equal.
- ❑ The equals() method compares two objects to determine whether the two object references point to the same object.

```
1. class IfStudentEqual {
2.     public static void main(String[] args) {
3.         // Create two objects
4.         Object Student_1 = new Student("Zhangsan",19);
5.         Object Student_2 = new Student("Lisi",18);
6.
7.         // Different objects, with different memory addresses, are not equal
8.         System.out.println(Student_1.equals(Student_2));
9.
10.        // Object references, with the same memory addresses, are equal
11.        Object Student_3 = Student_1;
12.        System.out.println(Student_1.equals(Student_3));
13.    }
14. }
```



super keyword

- ❑ The functionality of the super keyword:
 - Explicitly call the parent class constructor in the constructor of the child class.
 - Access member methods and variables of the parent class.



Super calls the parent class constructor

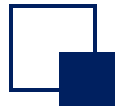
- ❑ The `super` keyword can explicitly call the constructor of the parent class in the constructor of the subclass, in a basic format such as: `super(parameters);`
 - where parameter-list specifies all the parameters in the parent class constructor.

```
1. public class Person {  
2.     public Person(String name, int age) {  
3.     }  
4. }  
5. public class Student extends Person {  
6.     public Student(String name, int age, String school) {  
7.         super(name, age); // Call a constructor with 2  
           parameters in the parent class  
8.     }  
9. }
```



Super access to the parent class member

- ❑ When a member variable or method of a child class has the same name as the parent class, for example, the child class overrides a method of the parent class, we can use the super keyword to call the method in the parent class.
- ❑ Accessing members in a parent class using super is similar to the use of the this keyword, except that it references the parent class of the child class, syntactically like : `super.X`
 - where X is a property or method in the parent class.



Super access to parent class member variables

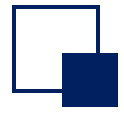
- ❑ In the example on the right, both the parent and child classes have a member variable age. We can use the super keyword to access the age variable in the Person class.

```
1. class Person {
2.     int age = 39;
3. }
4. class Student extends Person {
5.     int age = 18;
6.     void display() {
7.         System.out.println("Student Age: " + super.age);
8.     }
9. }
10. class Test {
11.     public static void main(String[] args) {
12.         Student stu = new Student();
13.         stu.display();
14.     }
15. }
```

output?



Student Age :
39



Super access to parent class member method

- When both the parent and child classes have the same method name, you can use the super keyword to access the parent class's methods.

```
1. class Person {  
2.     void getTarget() {  
3.         System.out.println("A good life");  
4.     }  
5. }  
6. class Student extends Person {  
7.     void getTarget() {  
8.         System.out.println("High scores");  
9.     }  
10.    void display() {  
11.        getTarget()  
12.        super.getTarget();  
13.    }  
14. }
```

Call getTarget() from
Student Class

Call getTarget() from
Person Class



Quiz

```
class Animal {
    public String name = "Animal";

    public Animal() {
        System.out.println("Animal()");
    }

    public void speak() {
        System.out.println("Animal.speak");
    }
}
```

```
class Dog extends Animal {
    public String name = "Dog";

    public Dog() {
        super(); // calls superclass constructor
        System.out.println("Dog()");
    }

    @Override
    public void speak() {
        System.out.println("Dog.speak");
    }

    public void demo() {
        System.out.println(super.name + " | " + this.name); // field hiding
        super.speak(); // call superclass version
        this.speak(); // call subclass version
    }
}

public class SuperKeywordQuiz {
    public static void main(String[] args) {
        Dog d = new Dog();
        d.demo();
    }
}
```



Quiz

```
class Animal {
    public String name = "Animal";

    public Animal() {
        System.out.println("Animal()");
    }

    public void speak() {
        System.out.println("Animal.speak");
    }
}

class Dog extends Animal {
    public String name = "Dog";

    public Dog() {
        super(); // calls superclass constructor
        System.out.println("Dog()");
    }

    @Override
    public void speak() {
        System.out.println("Dog.speak");
    }

    public void demo() {
        System.out.println(super.name + " | " + this.name); // field hiding
        super.speak(); // call superclass version
        this.speak(); // call subclass version
    }
}

public class SuperKeywordQuiz {
    public static void main(String[] args) {
        Dog d = new Dog();
        d.demo();
    }
}
```

Animal()
Dog()
Animal | Dog
Animal.speak
Dog.speak



Outline

- Inheritance
- Interfaces and abstract classes
- Polymorphism and Override
- Java multi-inheritance issues
- Superclass vs. super keyword
- **Inheritance framework for exceptions**



What is an exception?

- ❑ When a program is running, an unexpected event occurs that prevents the program from executing as the programmer intended, which is an exception.
 - Scenario 1: The user wants to enter an int age, but the user enters abc;

```
1.String s = "abc";    // Let's say the user enters ABC  
2.int n = Integer.parseInt(s); // NumberFormatException!
```

- Scenario 2: The program wants to read and write the contents of a file, but the user has deleted it.

```
1.// The user deleted the file:  
2.String t = readFile("C:\\abc.txt"); // FileNotFoundException!
```

Java exception handling mechanism

- ❑ Java has a built-in exception handling mechanism that always uses exceptions to indicate errors.
- ❑ An exception is a kind of class, so it carries type information of its own.
- ❑ There are three types of exceptions:
 - **Runtime Exceptions:** Runtime exceptions are exceptions that can be avoided by programmers. In contrast to inspecting exceptions, runtime exceptions can be ignored at compile time.
 - **Other Exceptions:** The most representative exceptions are those caused by user errors or problems that cannot be foreseen by the programmer. For example, when you want to open a file that doesn't exist, an exception occurs, and these exceptions cannot be simply ignored at compile time.
 - **Error:** An error is not an exception, but a problem that is out of the programmer's control. Errors are usually ignored in the code. For example, when a stack overflow occurs, an error occurs that is not checked even at compilation.

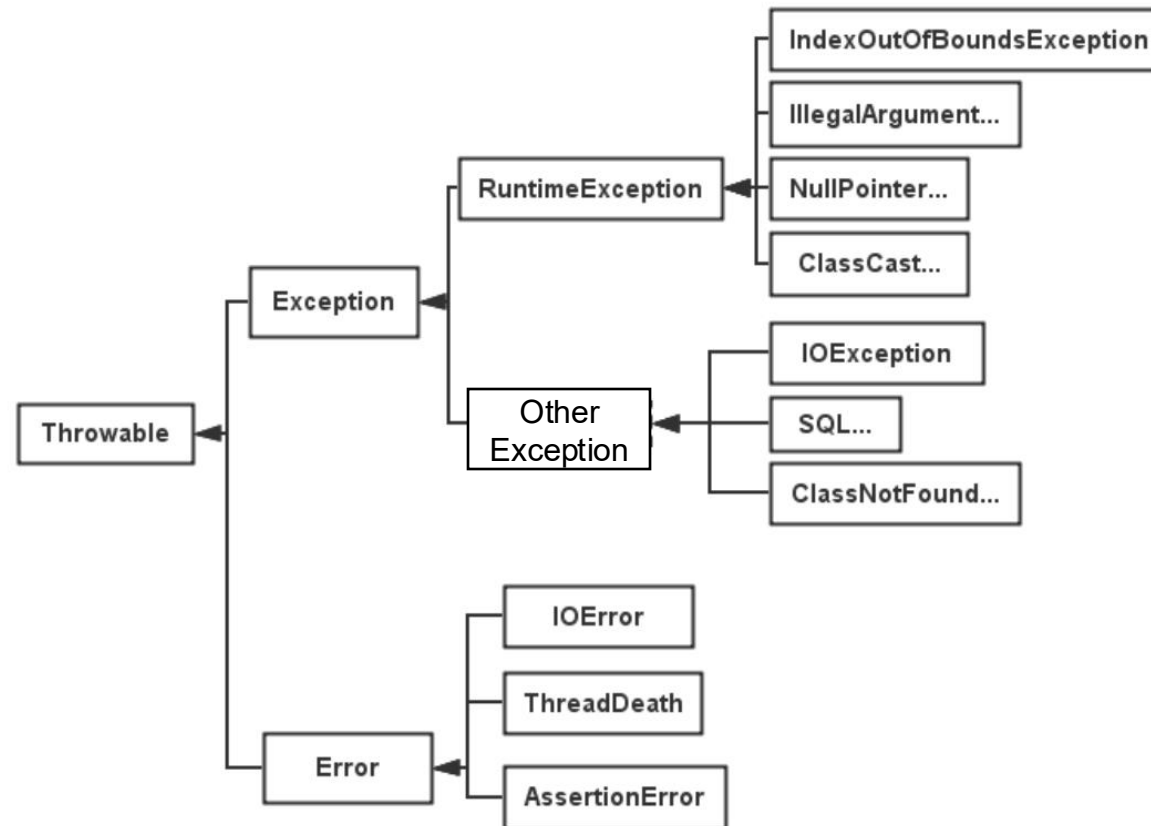


Benefits of using exceptions

- ❑ Tell us where the program went wrong.
- ❑ Allow an otherwise interrupted program to continue running in an appropriate manner, or exit.
- ❑ Save the user's current operations and release the occupied resources.
- ❑ Reduce code complexity and separate errors from business code.

Inheritance framework for exceptions

- ❑ From the inheritance relationship, it can be seen that Throwable is the root of the anomaly system, which inherits from Object.
- ❑ All exception classes are inherited from the java.lang.Exception class.



Inheritance framework for exceptions

- ❑ Throwable has two systems: Error and Exception, with Error indicating a serious error that the program is generally powerless to do.
- ❑ Exceptions, on the other hand, are runtime errors that can be caught and handled. The Exception class is further divided into two main categories:
 - RuntimeException and its subclasses;
 - Other Exception (including SQLException、IOException、XMLParseException, etc)



Design custom exceptions

- ❑ Exceptions can be customized in Java. Be careful when writing custom exception classes:
 - All exception classes must be subclasses of Throwable.
 - To customize a checked exception class, you need to inherit from the Exception class.
 - To customize a runtime exception class, you need to inherit the RuntimeException class.
- ❑ You can customize the exception class like the following:

```
// Customize the exception class
class MyException extends Exception{

}
```



Custom exception instances

- ❑ Write a program to check whether the grades included in the system are in the [0,100] range.
- ❑ This first creates an exception class, MyException, and provides two constructors.
 - The default construction method without parameters.
 - Receives a custom exception message as a string and passes that message to the superclass's constructor

```
1. public class MyException extends Exception {  
2.     public MyException() {  
3.         super();  
4.     }  
5.     public MyException(String str) {  
6.         super(str);  
7.     }  
8. }
```

Custom exception instances

- ❑ Next, create a test class and call the custom exception class.

```
1. import java.util.InputMismatchException;
2. import java.util.Scanner;

3. public class exception_eg {
4.     public static void main(String[] args) {
5.         int score;
6.         Scanner input = new Scanner(System.in);
7.         System.out.println("Please enter your test scores: ");
8. ....
```



Custom exception instances

```
9.      try {
10.          score = input.nextInt();    // Get grades
11.          if(score < 0) {
12.              throw new MyException("The grade is negative! ");
13.          } else if(score > 100) {
14.              throw new MyException("You are too excellent! ");
15.          } else {
16.              System.out.println("Your grades are:: "+score);
17.          }
18.      } catch(InputMismatchException e1) {
19.          System.out.println("The grade is not a number! ");
20.      } catch(MyException e2) {
21.          System.out.println(e2.getMessage());
22.      }
23. ....
```



Custom exception instances

- ❑ Run the program, when the score entered by the user is negative, the MyException custom exception will be thrown, the code in the second catch statement block will be executed, and the exception information will be printed. The results of the program are shown below.

```
Please enter your grade:  
-2  
Negative grades!
```



Quiz

```
// Step 1: Define a custom exception
class InsufficientFundsException extends ? {
    public InsufficientFundsException(String message) {
        super(message);
    }
}

// Step 2: Define the BankAccount class
class BankAccount {
    private double balance;

    public BankAccount(double balance) {
        this.balance = balance;
    }

    public void withdraw(double amount) throws InsufficientFundsException {
        if (amount > balance) {
            throw new InsufficientFundsException("Insufficient funds: balance is " + balance);
        }
        balance -= amount;
        System.out.println("Withdrawal successful. Remaining balance: " + balance);
    }
}

// Step 3: Main class to test
public class ExceptionQuiz {
    public static void main(String[] args) {
        BankAccount account = new BankAccount(100.0);
        try {
            account.withdraw(80.0);
            account.withdraw(50.0);
        } catch (InsufficientFundsException e) {
            System.out.println("Exception caught: " + e.getMessage());
        }
    }
}
```



Quiz

```
// Step 1: Define a custom exception
class InsufficientFundsException extends Exception {
    public InsufficientFundsException(String message) {
        super(message);
    }
}

// Step 2: Define the BankAccount class
class BankAccount {
    private double balance;

    public BankAccount(double balance) {
        this.balance = balance;
    }

    public void withdraw(double amount) throws InsufficientFundsException {
        if (amount > balance) {
            throw new InsufficientFundsException("Insufficient funds: balance is " + balance);
        }
        balance -= amount;
        System.out.println("Withdrawal successful. Remaining balance: " + balance);
    }
}

// Step 3: Main class to test
public class ExceptionQuiz {
    public static void main(String[] args) {
        BankAccount account = new BankAccount(100.0);
        try {
            account.withdraw(80.0);
            account.withdraw(50.0);
        } catch (InsufficientFundsException e) {
            System.out.println("Exception caught: " + e.getMessage());
        }
    }
}
```

Withdrawal successful.
Remaining balance: 20.0

Exception caught: Insufficient
funds: balance is 20.0



Outline

- **Inheritance**
- **Interfaces and abstract classes**
- **Polymorphism and Override**
- **Java multi-inheritance issues**
- **Superclass vs. super keyword**
- **Inheritance framework for exceptions**